



Build it together.

Today we combine **TF-IDF (Day 13)**, regex text cleaning (**Day 11**), and Streamlit UI (**Days 4-5**) into one app — a **Smart Notes Summarizer**. Paste long lecture notes, get the 5 most important sentences + key terms + reading time.



PROGRAM

Python Internship

LED BY

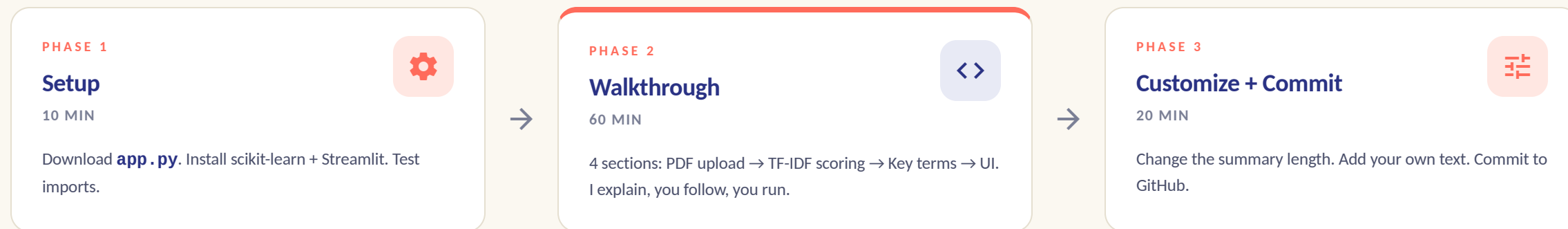
Zlaark

SESSION

14 of 25

Pre-shared file. Walk through together. Run after each section.

The complete app.py is in the shared folder. Download it, open in VS Code. I explain each section on the projector. You follow along in your own copy. Run after each section to see it work.



THE APP

Paste lecture notes (or upload a .txt file). Get: top 5 most important sentences (by TF-IDF), top 10 key terms, reading time, word count. Like a mini TL;DR for lectures.

• THE TARGET • WHAT WE'RE BUILDING

A Streamlit app that summarizes your lecture notes.

Paste text (or upload a .txt file). TF-IDF scores every sentence. The top 5 become your summary. Plus: key terms, reading time, and a word frequency chart.

localhost:8501 – Smart Notes Summarizer

Smart Notes Summarizer

PASTE YOUR NOTES HERE

Machine learning is a subset of artificial intelligence. It enables systems to learn from data. Models improve through training without explicit programming...

SUMMARY LENGTH: 5 SENTENCES

Summarize

SUMMARY

1. Machine learning is a subset of artificial intelligence.
2. It enables systems to learn from data.
3. Models improve through training without explicit programming.
4. Common algorithms include decision trees and neural networks.
5. Data quality directly affects model accuracy.

KEY TERMS

machine learning data models training ai algorithms neural accuracy quality

WORDS
450

READING
2 min

SENTENCES
18

WORD IMPORTANCE CHART

THE FEATURES

- 1 Text input**
Paste notes or upload .txt file
- 2 TF-IDF scoring**
Scores each sentence by word importance (Day 13)
- 3 Summary**
Top N sentences by score, in original order. Slider for N.
- 4 Key terms**
Top 10 words by TF-IDF weight
- 5 Stats + chart**
Word count, reading time, frequency chart (Day 7)

• STEP 0 • SETUP (10 MIN)

Download **app.py**. Install. Test imports.

The complete file is in the shared folder. Download it, install dependencies, verify it runs.

● setup.sh

```
# 1. Activate your venv
$ source .venv/bin/activate

# 2. Install (most are already installed)
$ pip install streamlit scikit-learn

# 3. Download app.py from shared folder
#    (or copy from instructor's USB)

# 4. Open in VS Code
$ code app.py

# 5. Test imports – check the top of app.py has:
import streamlit as st
from sklearn.feature_extraction.text import TfidfVectorizer
import re
import pandas as pd

# 6. Run it
$ streamlit run app.py
#    Browser opens. You see "Smart Notes Summarizer"
```

✓ venv activated · ✓ pip install succeeded · ✓ app.py downloaded & open · ✓ All 4 imports work · ✓ streamlit run opens browser · ✓ You see the app title

• BUILD STEP 1 OF 4 • TEXT INPUT + SPLITTING (15 MIN)

Paste text. Split into sentences. Clean it.

Day 11 skills. The user pastes notes. We split into sentences using regex. Clean the text (lowercase, remove extra spaces).

● app.py (Section 1)

SECTION 1

```
import streamlit as st
import re
from sklearn.feature_extraction.text import TfidfVectorizer
import pandas as pd

st.title("Smart Notes Summarizer")
st.write("Paste your lecture notes. Get the most important sentences instantly.")

# --- SECTION 1: Text Input + Sentence Splitting ---
text = st.text_area(
    "Paste your notes here",
    height=200,
    key="notes_input",
    placeholder="Paste lecture notes, an article, or any long text..."
)

# Slider for summary length
num_sentences = st.slider("Summary length (sentences)", 3, 10, 5, key="slider")

if text.strip():
    # Split text into sentences using regex
    # Day 11: regex for sentence boundaries
    sentences = re.split(r'(?<=[!?!])\s+', text.strip())
    sentences = [s.strip() for s in sentences if len(s.strip()) > 10]

    st.write(f"Found {len(sentences)} sentences.")
```

🚩 CHECKPOINT

After running Section 1:

- ✓ Text area appears
- ✓ Slider appears (3-10 sentences)
- ✓ Paste text → "Found N sentences" appears
- ✓ Sentences are split correctly (not on "Mr." or "Dr.")

DAY 11 CONNECTION

re.split() with a lookahead regex for sentence boundaries. Same regex skills from the regex deep-dive.

• BUILD STEP 2 OF 4 • TF-IDF SCORING (15 MIN)

Score each sentence. Pick the top N.

Day 13 skills. TF-IDF vectorizes all sentences. Each sentence gets a score = sum of its word weights. The top N sentences become the summary.

● app.py (add Section 2)

SECTION 2

```
# --- SECTION 2: TF-IDF Scoring ---
# Day 13: TfidfVectorizer scores words by importance
vectorizer = TfidfVectorizer(stop_words='english')
tfidf_matrix = vectorizer.fit_transform(sentences)

# Score each sentence = sum of its word TF-IDF weights
import numpy as np
sentence_scores = np.array(tfidf_matrix.sum(axis=1)).flatten()

# Rank sentences by score (highest = most important)
ranked_indices = np.argsort(sentence_scores)[::-1]

# Pick top N sentences, but keep them in ORIGINAL order
top_indices = sorted(ranked_indices[:num_sentences])
summary = [sentences[i] for i in top_indices]

# Show the summary
st.subheader("Summary")
for i, sentence in enumerate(summary, 1):
    st.write(f"{i}. {sentence}")
```

🚩 CHECKPOINT

After running Section 2:

- ✓ Paste text → summary appears
- ✓ Summary has the correct number of sentences (matches slider)
- ✓ Sentences are in ORIGINAL order (not ranked order)
- ✓ Changing the slider changes the summary length
- ✓ stop_words='english' removes "the", "is", "a"

DAY 13 CONNECTION

TfidfVectorizer + fit_transform. Same 5-line pattern from Day 13, applied to sentences instead of documents.

• BUILD STEP 3 OF 4 • KEY TERMS + STATS (15 MIN)

Show key terms. Show stats. Show a chart.

Day 13 + Day 7 skills. Extract the top 10 words by TF-IDF weight. Calculate word count + reading time. Show a frequency bar chart.

● app.py (add Section 3)

SECTION 3

```
# --- SECTION 3: Key Terms + Stats + Chart ---
# Day 13: get_feature_names_out() for vocabulary
feature_names = vectorizer.get_feature_names_out()
word_weights = np.array(tfidf_matrix.sum(axis=0)).flatten()

# Top 10 key terms by TF-IDF weight
top_term_indices = np.argsort(word_weights)[::-1][:10]
key_terms = [(feature_names[i], word_weights[i]) for i in top_term_indices]

st.subheader("Key Terms")
cols = st.columns(5)
for i, (term, weight) in enumerate(key_terms):
    cols[i % 5].metric(term, f"{weight:.2f}")

# Stats
word_count = len(text.split())
reading_time = max(1, round(word_count / 200)) # 200 wpm

col1, col2, col3 = st.columns(3)
col1.metric("Words", word_count)
col2.metric("Reading time", f"{reading_time} min")
col3.metric("Sentences", len(sentences))

# Day 7: bar chart of word frequencies
word_freq = pd.DataFrame(key_terms, columns=["word", "weight"])
st.subheader("Word Importance Chart")
st.bar_chart(word_freq.set_index("word")["weight"])
```

🚩 CHECKPOINT

After running Section 3:

- ✓ Key terms appear as 10 metric cards
- ✓ Stats show: word count, reading time, sentence count
- ✓ Bar chart shows word importance
- ✓ Chart updates when you change the text
- ✓ Key terms are NOT stop words ("the", "is" are removed)

DAY 7 + DAY 6 CONNECTION

`st.bar_chart` from Day 7.
`pandas.DataFrame` from Day 6.

• BUILD STEP 4 OF 4 • POLISH + GITHUB (15 MIN)

Polish. Commit. You're done.

Add a sidebar, handle empty input, add a footer. Then commit to GitHub.

🔧 POLISH CHECKLIST

1 Add a sidebar

```
with st.sidebar: st.write("**Smart Notes Summarizer**\nPaste notes → get summary")
```

2 Handle empty input

```
if not text.strip(): st.info("Paste your notes above to begin.")
```

3 Add a footer

```
st.divider(); st.caption("Built with Python, Streamlit, scikit-learn. TF-IDF powered summarization.")
```

📦 COMMIT TO GITHUB

• git-commands.sh

```
$ cd day14
$ git init
$ git remote add origin https://github.com/YOURNAME/day14-notes-summarizer.git

# .gitignore
$ echo "__pycache__/"
.venv/
*.pyc" > .gitignore

$ git add app.py .gitignore
$ git commit -m "Smart Notes Summarizer - TF-IDF + Streamlit"
$ git push -u origin main
```

WHAT TO TEST Paste real lecture notes. Does the summary make sense? Are the key terms relevant? Does the chart show the right words? If yes → you're done. 🎉

You just built an AI-powered summarizer.

In **80 minutes**. Using **TF-IDF (Day 13)**, **regex (Day 11)**, **Streamlit (Days 4-5)**, **pandas (Day 6)**, and **charts (Day 7)**. Tomorrow we recap everything before the group project starts.

THE PIPELINE

What You Built

- ✓ Text input → regex sentence splitting (**Day 11**)
- ✓ TF-IDF vectorization → sentence scoring (**Day 13**)
- ✓ Top N selection → summary in original order
- ✓ Key terms extraction → `get_feature_names_out` (**Day 13**)
- ✓ Stats → word count, reading time
- ✓ Bar chart → `st.bar_chart` (**Day 7**)
- ✓ Streamlit UI → `text_area`, `slider`, `metric`, `columns` (**Days 4-5**)
- ✓ Git commit → GitHub (**Day 3**)

CHECKLIST

Before Tomorrow

- ✓ Test with real lecture notes
- ✓ Commit to GitHub (**green square!**)
- ✓ Try different text — does the summary change?
- ✓ Email `hello@zlaark.com` if something broke
- ✓ Rest — tomorrow is recap, then Day 16: group project

DAYS 15-16

What's Next

-  **DAY 15**
Recap & doubt clearing
-  **DAY 16**
Group project begins
-  **CAPSTONE**
10 days of building. 1 demo.



See you on Day 15.